Warmup Lab

This warmup lab contains six Java programming challenges. Complete each one and upload it to Brightspace (this lab will not count toward the final grade).

These are taken from CodingBat; you are encouraged to complete more of these problems on your own to refresh your Java.

How the lab is organized

Each challenge lives in its own folder, `ch1` through `ch6`. Every folder contains a `Main.java` with:

- a `main` method that asks you for an input and runs your solution on it, and
- a method stub marked `//TODO` that you need to complete.

Compile and run a challenge from inside its folder:

```
javac Main.java
java Main
```

The scaffold compiles and runs before you have written anything, because the stub simply returns a placeholder value. This is on purpose: you should be able to compile and run your program at every step, not only once it is finished.

Every folder also contains a `Test.java`, described in the next section.

The challenges that take an array read it as one line of values separated by spaces, so `1 2 3` becomes the array `{1, 2, 3}` and an empty line becomes an empty array. The `parseInts` and `parseStrings` methods that do this are written for you.

Testing your code

Testing your function is usually the best way to catch bugs in your code before someone else does. No test suite can catch every possible bug, but thorough testing can prevent most common errors.

Start testing your solution to Challenge 1, typing one input at a time and re-running the program for each one. You will notice fairly quickly that this gets tedious, and tedious things are the things people quietly stop doing.

Each folder also holds a `Test.java`. It is a small class whose `main` runs your solution on a whole list of inputs and reports whether each one gave the answer you expected. Run it from inside the challenge folder:

```
javac Test.java
java Test
```

You do not need to compile `Main.java` separately; `javac` finds it for you.

The `Test.java` in Challenge 1 is filled in as a worked example. Each of its lines looks like this:

```
check( "abcXXXabc", 1 );
```

That is an input and the answer you expect from it. The `check` method at the bottom of the file calls `countTriples` for you, so a test case is never more than one short line.

Look closely at the three cases that expect 0. Those pass even against the placeholder that returns 0, before any real solution has been written. They are not bad tests, and they are certainly not evidence that the program works. A test that passes tells you only that it did not catch a bug this time. This cuts both ways all semester: a failing test proves something is wrong, while a passing test never proves anything is right.

For Challenges 2 to 6 the `Test.java` is there with a single case already written. Add one line for each further case you want to check. Each challenge has its own `check`, written for that challenge's function, so the arguments you pass are simply that function's inputs followed by the answer you expect.

Finally, a function only has to behave sensibly on the inputs its assignment promises it. Challenge 6 tells you that each array is sorted and contains at least `n` elements, so you do not need to test it on unsorted arrays or on an `n` that is too large. Saying what a function does when its assumptions are broken needs exceptions, which are coming in the next lecture.

Each challenge below ends with a list of cases that are worth thinking about. They are mostly "exceptional" cases, but don't forget to test your function on some "normal" input as well!

Challenge 1 — `countTriples`

Write a function that given a string, counts the number of triples in that string. A triple is any character that appears three times in a row. For example:

- `countTriples("abcXXXabc")` should return 1 because there is a single triple of `"XXX"`
- `countTriples("xxxabcyyyydef")` should return 3 because we have one triple of `"xxx"` and two triples of `"yyy"` (since there are a total of four y's in the given string).

Testing. Does your function work if you give it:

- an empty string?
- a string with a triple at the very start?
- a string with a triple at the very end?
- a non-empty string with no triples?
- a string with quadruples or longer sequences?

Challenge 2 — `sumDigits`

Write a method that computes and returns the sum of all the digits in a given string. Characters that are not digits are ignored. For example:

- `sumDigits("a1b")` should be 1
- `sumDigits("a1b2c")` should be 3
- `sumDigits("a12b")` should be 3

Complete the `sumDigits` method in `ch2/Main.java`. You may modify `main` as you wish.

Hint. The characters in most programming languages are represented using their numerical code. Your program can take advantage of that fact. Take a look at the code below and see if you can use it in your solution:

```
public class Example {
    public static void main( String [] args ) {
        System.out.println("digit   numerical code    value  ");
        for (char digit = '0'; digit <= '9'; digit++) {

            System.out.printf( "  '%c'  %10d %12d\n",
                    digit,              // print the digit which is a character
                    (int) digit,        // print the numerical code when char is cast to int
                    (int) (digit-'0')   // print the value that is represented by that digit
            );
        }
    }
}
```

Testing. Does your function work if you give it:

- an empty string?
- a string that contains no digits at all?
- a string that is nothing but digits?
- a string with digits at the very start and at the very end?
- a string containing `/` and `:`? (These are the characters just below `'0'` and just above `'9'` in the code table, exactly what a boundary that is off by one will let through.)

Challenge 3 — `divideSelf`

For this challenge we will say that an integer divides itself if each digit in that integer divides the integer itself evenly (that is, without any remainder). Zero does not divide any number, so any number that contains the digit zero does not divide itself. For example:

- `divideSelf(128)` returns true, since 1, 2, and 8 all divide 128 without remainder
- `divideSelf(13)` returns false, because 13/3 is 4 with remainder 1, so 3 does not divide 13 evenly
- `divideSelf(120)` returns false, because 0 does not divide any number
- `divideSelf(-128)` returns true, for the same reason `divideSelf(128)` does: whether a digit divides a number evenly does not depend on the sign

Hint. The operators `%` and `/` may come in handy for this problem. Recall that:

- `%` is the modulus operator: `128 % 10` gives us 8, the last digit of the number
- `/` is integer division when applied to integers: `128 / 10` gives us 12, the number without its last digit

Testing. Does your function work if you give it:

- a single-digit number, such as 7?
- a number containing a zero somewhere in the middle, such as 105?
- the number 0 itself?
- a number whose digits repeat, such as 111?
- a negative number, such as -128? (It should give the same answer as 128 — but check what `-128 % 10` actually gives you before assuming your digit loop handles it.)

Challenge 4 — `longestSpan`

Consider the leftmost and rightmost occurrence of some value in an integer array. The span of that value is the number of elements between these two occurrences (the number includes the value itself). A single value has a span of 1. The longest span is the largest span for any value in an array. The longest span is zero for an empty array.

Complete the `longestSpan` method in `ch4/Main.java` to compute the longest span for a given array. For example:

- `longestSpan([1, 2, 1, 2, 3])` returns 3, because the span of both 1 and 2 is 3
- `longestSpan([1, 2])` returns 1, because the span of both 1 and 2 is 1
- `longestSpan([1, 2, 3, 2, 1])` returns 5

Testing. Does your function work if you give it:

- an empty array?
- an array with a single element?
- an array in which every element is the same?
- an array in which every element is distinct?
- an array whose longest span ends at the very last element?
- an array containing negative numbers?

Challenge 5 — `balance`

Given an integer array, determine if the array can be split into a left part and a right part so that the sum of each part is the same. Your function should accept an array as its argument and return a boolean value indicating whether this is possible. Either part is allowed to be empty, so an empty array is balanced: both parts are empty, and both sums are 0. For example:

- `balance([1, 1, 1, 2, 1])` returns true, since we can split the array after index 2: `[1, 1, 1]` and `[2, 1]` both sum to 3
- `balance([1, 2, 3, 5])` returns false (note that if the sum of all the values is odd, the answer will always be false)
- `balance([10, 10])` returns true
- `balance([2, 2, 8])` returns false, because no split results in the two sums being equal

Testing. Does your function work if you give it:

- an empty array?
- an array with a single element?
- an array whose split point is at the very start or the very end?
- an array in which every element is zero?
- an array whose values sum to an odd number?
- an array containing negative numbers?

Challenge 6 — `merge`

Start with two arrays of strings that are each sorted in alphabetical order. The values within each array are unique, but the same value could appear in both arrays. We will assume that each array contains at least `n` elements. Write a function that, given two such arrays and the value `n`, creates and returns another array that

- contains the smallest `n` elements from the two arrays combined,
- is sorted, and
- contains unique elements.

For example:

- `merge(["a", "c", "k", "p"], ["c", "e", "m", "z"], 3)` returns `["a", "c", "e"]`
- `merge(["apple", "banana", "kiwi", "melon", "pear"], ["carrot", "cucumber", "kale", "lettuce", "tomato"], 4)` returns `["apple", "banana", "carrot", "cucumber"]`

Hint 1. If you are unfamiliar with the `String` class's `compareTo` method, you may want to look it up.

Hint 2. The most efficient solution will make a single pass through each array and should not need to sort anything.

Testing. Does your function work if you give it:

- `n` equal to 0?
- `n` equal to the full length of both arrays?
- two arrays with no values in common?
- two arrays that are identical to each other?
- two arrays where all `n` of the smallest values come from the same one?
- two arrays that share a value right at the cutoff, so that a duplicate would push a real answer out of the result?

For this challenge you do not need to test arrays that are unsorted, arrays that repeat a value within themselves, or an `n` larger than either array, because the assignment promises those will not happen. Once we have seen Java exceptions, you should also test that these inputs throw the right one.

The version of `check` that compares arrays uses `Arrays.equals`, not `==`. It is worth opening `Test.java` to see that, and asking yourself what `==` would have compared instead — the answer matters a great deal in this course.